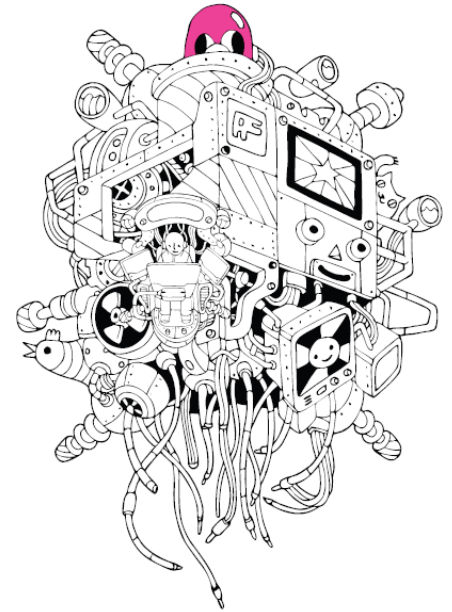


C programming



Chapter 02. Basic structure of the program

A collection of 15 cartoon illustrations of various objects, including a cat, a fish, a planet, a laptop, a game console, a die, a key, and a robot, arranged in a circular pattern.

100%

100

Understanding 'functions', the basic unit of the C language

✓ The basic unit of the C language is a function .

A C program is made by creating functions and deciding the order in which they run.

✓ Basic characteristics of functions

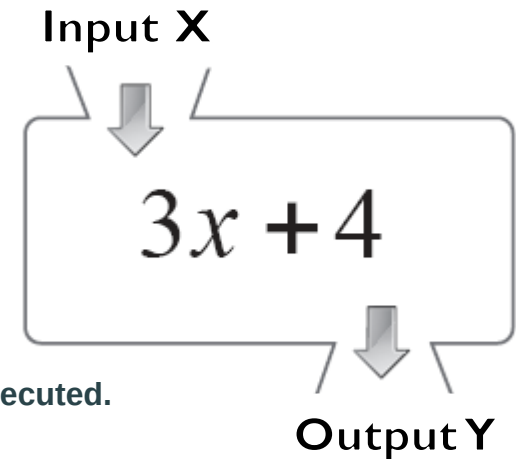
In mathematics, a function has inputs and outputs.

✓ Functions in the C language

Functions in the C language also have input and output.

✓ Summary of terms related to functions in C language

- **Function definition** : A function that has been created and can be executed.
- **Function call** : Running (executing) a function.
- **Argument passing** : Sending input values to a function when calling it.



The C language begins with functions and ends with functions.



In the Hello.c example, where is the function?

✓ Start of the program

C program starts when the **main** function is called.

✓ Which function is called first ?

The function named **main** is called first.

All C programs must define a function called **main** because it is the starting point of the program.

The program begins execution when the **main** function is automatically called.

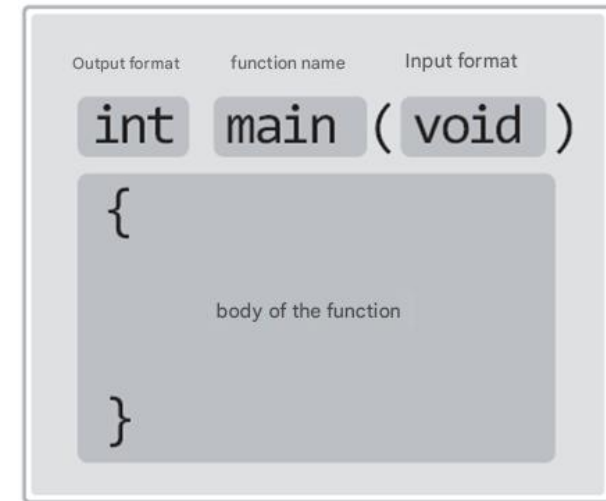
✓ Function features

A function's code is written inside curly brackets { }.

The code inside the curly brackets is called the **function body**.

✓ Three things that appear in functions in the C language

- **Function name** – The name used to call the function
- **Return type** – The type of value the function returns after execution.
- **Parameter type** – The types of input values passed to the function



```
int main(void)
{
    printf("Hello world! \n");
    return 0;
}
```

Run sequentially



semicolon

✓ A semicolon character ; is added to the end of a statement within a function .

A semicolon (;) is placed at the end of a statement in a function. It shows the end of a statement.

✓ Are ten lines equal to ten statements?

Ten lines of code do not always mean ten statements. One statement can be written on several lines.

Also, several statements can be written on one line. A line break does not mean the statement has ended.

✓ Writing style

It is common and clearer to write one statement per line.

The three main functions below are exactly the same program. Different line breaks do not change how the program works.

```
int main(void)
{
    printf("Hello world!\n"); return 0;
}
```

```
int main(void)
{
    printf("Hello world! \n");
    return 0;
}
```

```
int main(void) { printf("Hello world!\n"); return 0; }
```



Detailed analysis of source code

```
#include <stdio.h> Header file declaration
```

```
int main(void)
```

```
{ A call statement for a function  
seen for the first time
```

```
    printf("Hello world! \n");
```

```
    return 0;
```

```
}
```

✓ Standard function

A function that is already created and provided by default !

printf function is one of standard functions.

✓ Standard library

It refers to a collection of standard functions.

That is , the printf function is part of the standard library.

```
#include <stdio.h>
```

- This line tells the program to use the stdio.h file.
- This means to place the contents of the stdio.h file in this location.
- The stdio.h file contains information for standard input and output functions.

```
printf("Hello world! \n");
```

- This line calls the printf function.
- The argument is the string "Hello world! \n".
- Arguments are written inside parentheses ().
- \n means “new line” (move to the next line).

```
return 0;
```

- This line ends the function.
- It sends the value 0 back to the operating system.
- 0 usually means the program ended successfully.



C programming



Chapter 02-2. A complete program with
comments

The Importance of Comments

✓ What are comments?

Comments are notes written inside the source code. The computer does not read or run comments.

So comments do not change how the program works.

✓ Why do we need comments?

Code is not always easy to understand. So we add comments to explain the code.

Comments help other people understand the program. They also help the person who wrote the code.

In short, comments are very important.

✓ Block-level comments

Comment out a single line

```
/* 주석처리 된 문장 */
```

```
/*  
    주석처리 된 문장1  
    주석처리 된 문장2  
    주석처리 된 문장3  
*/
```

Comment out multiple lines

✓ Line-level comments

Commenting on a single line

```
// 주석처리 된 문장1  
// 주석처리 된 문장2  
// 주석처리 된 문장3
```

How to write comments can be different for each project. Usually, the team members discuss and decide together how to write comments.



Example of comment processing

```
/*
Title: Print Hello world
Function: output of string
File name: HelloComment.c
Last modified: July 15, 2014
Written by: Seongwoo Yoon
*/
#include <stdio.h> // Header file declaration

int main(void)      // Start of main function
{
    /*
    This function outputs a single string.
    The string is output to the monitor.
    */
    printf("Hello world! \n"); // Print string
    return 0;                 // return 0
} // End of main function
```

Over-processed comments

(even comments that are too much are not good)!

This is just an example to introduce how to add comments .



Cautions for commenting

```
1.  /*
2.     주석처리 된 문장1
3.     /* 단일 행 주석처리 */
4.     주석처리 된 문장2
5.  */
```

*Incorrectly placed comment
(error occurs during compilation)*

```
1.  /*
2.     주석처리 된 문장1
3.     // 단일 행 주석처리
4.     주석처리 된 문장2
5.  */
```

*Well-executed comments
(no errors during compilation)*

Sometimes comments can overlap. However, block comments cannot overlap.

Line comments can overlap in some cases But block comments (/* */) cannot be placed inside another block comment.



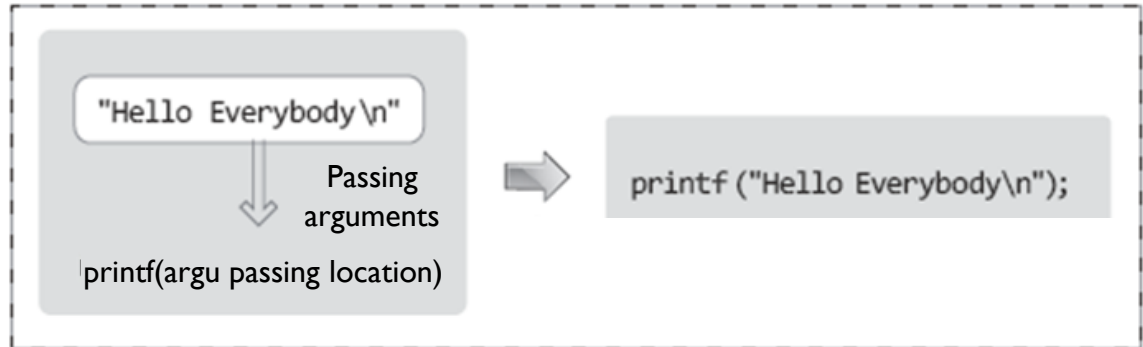
C programming



Chapter 02-3. Basics of the printf function understanding

Outputting integers with %d

```
int main(void)
{
    printf("Hello Everybody\n");
    printf("%d\n", 1234);
    printf("%d %d\n", 10, 20);
    return 0;
}
```



Execution results

```
Hello Everybody
1234
10 20
```

%d

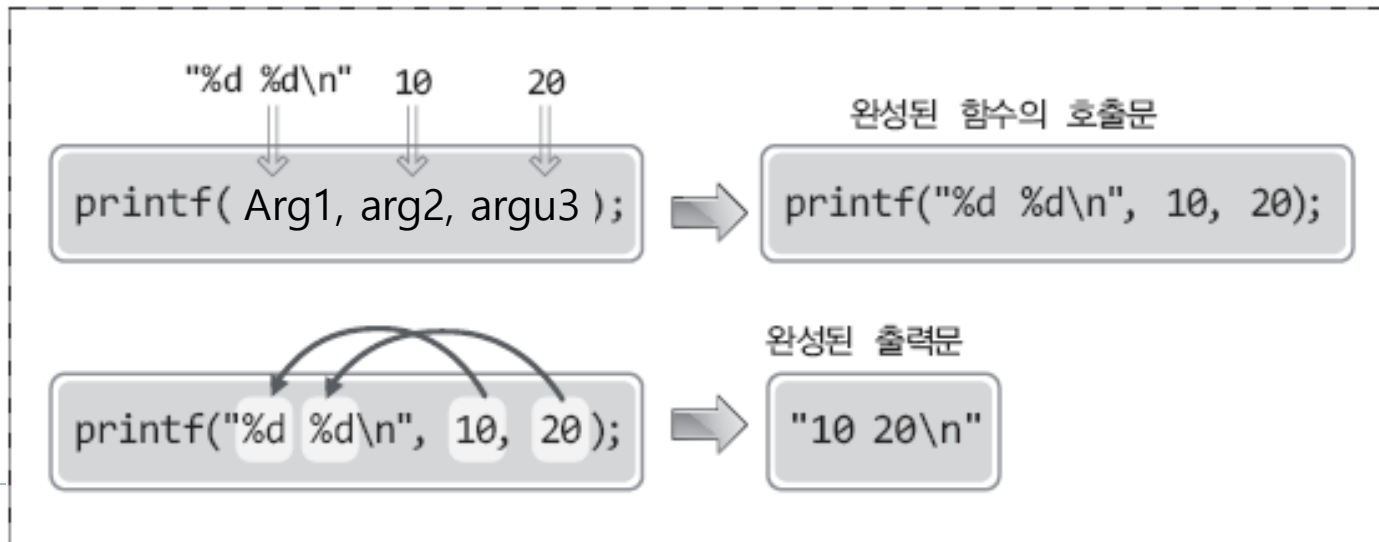
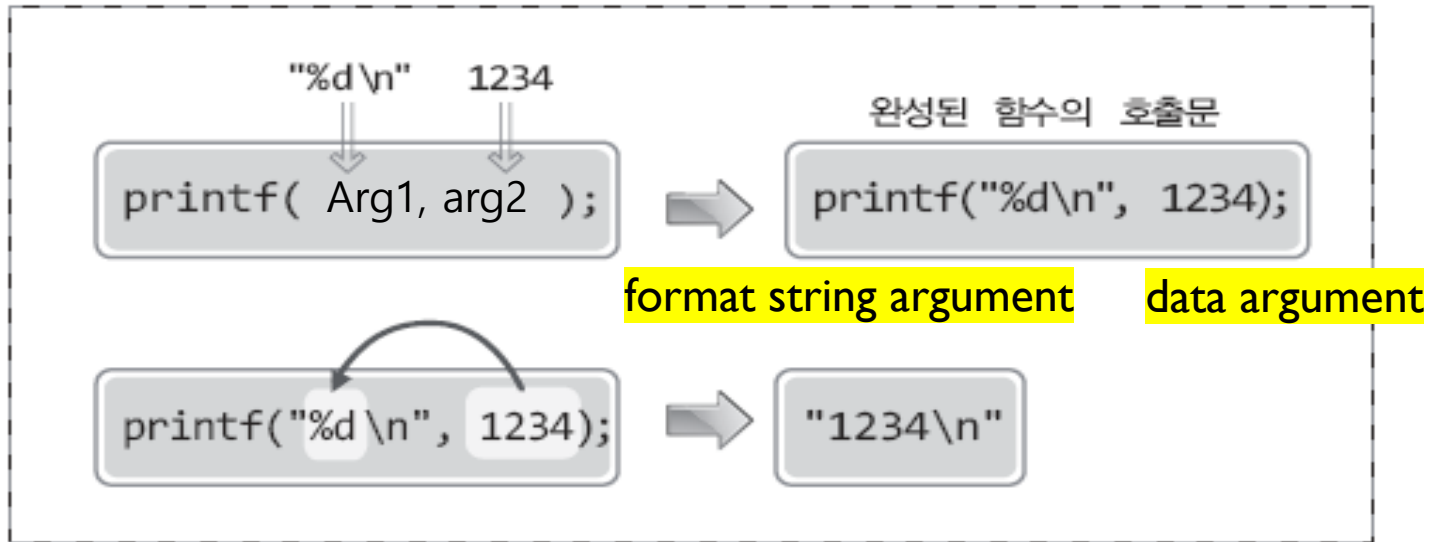
- **%d is a "format specifier".**
- **A format specifier tells the program how to display the output.**
- **%d is used to print a signed decimal integer (a whole number).**

Where do we put the values?

- **The values are written after the string (after the double quotation marks).**
- **They are separated by commas.**
- **If the format character %d appears twice, the output target must also appear twice .**

\n is an escape sequence or It is called a special character and is used to mean a new line.

%d

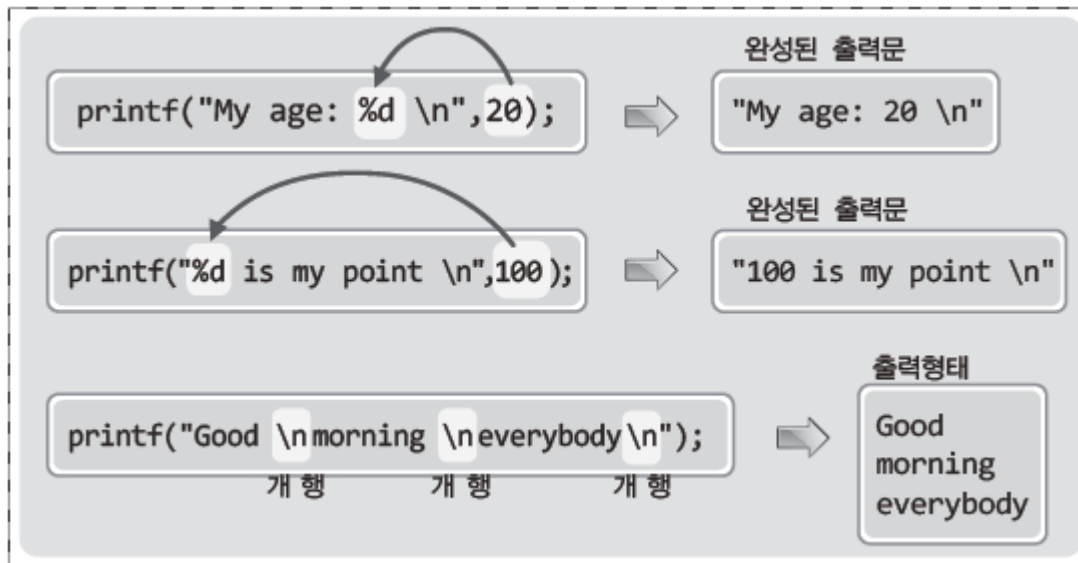


It is possible to combine various output formats .

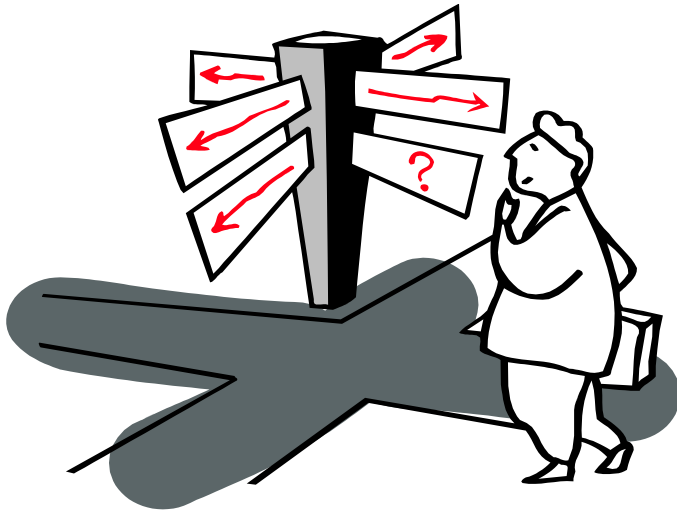
```
int main(void)
{
    printf("My age: %d \n", 20);
    printf("%d is my point \n", 100);
    printf("Good \nmorning \neverybody\n");
    return 0;
}
```

Execution results

```
My age: 20
100 is my point
Good
morning
everybody
```



Later, you will learn more format specifiers.
Then, you will be able to print output in many different ways.



Chapter 02 That's it.

Do you have any questions ?